

P1687.1 Modelling of a P1687.2 System

M. Portolan

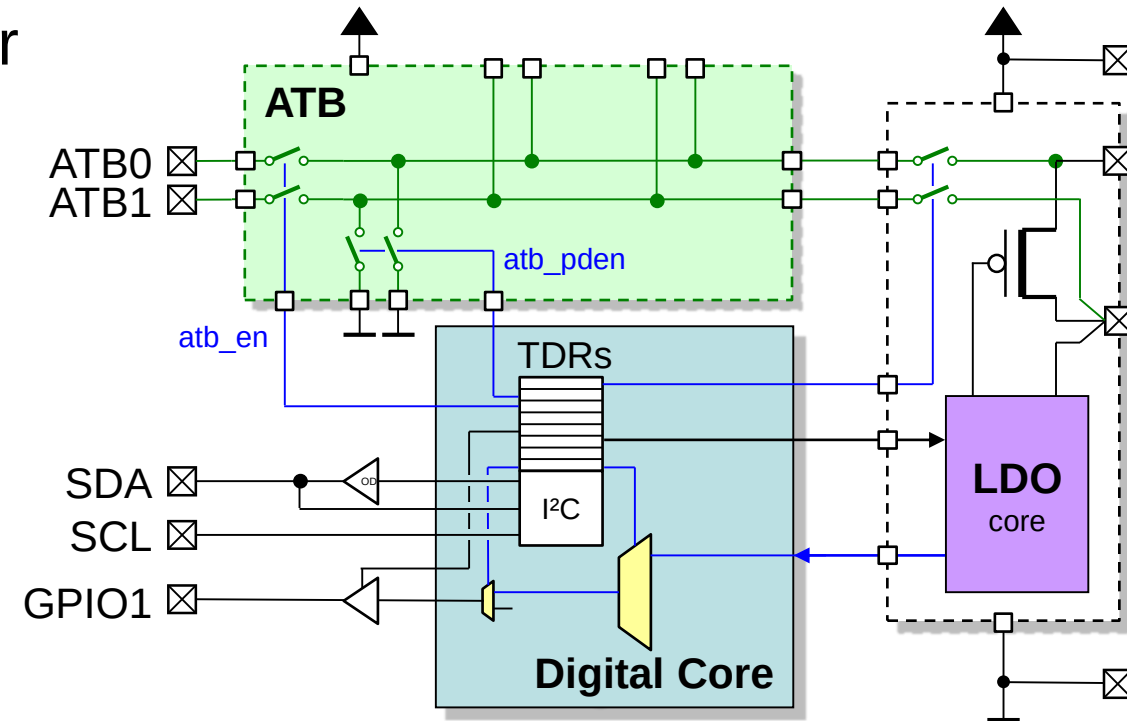
Univ Grenoble Alpes, CNRS, Grenoble INP, TIMA

P1687.2 Example System

Mixed-Signal example with Big-A: Power Management IC (PMIC) from Renesas

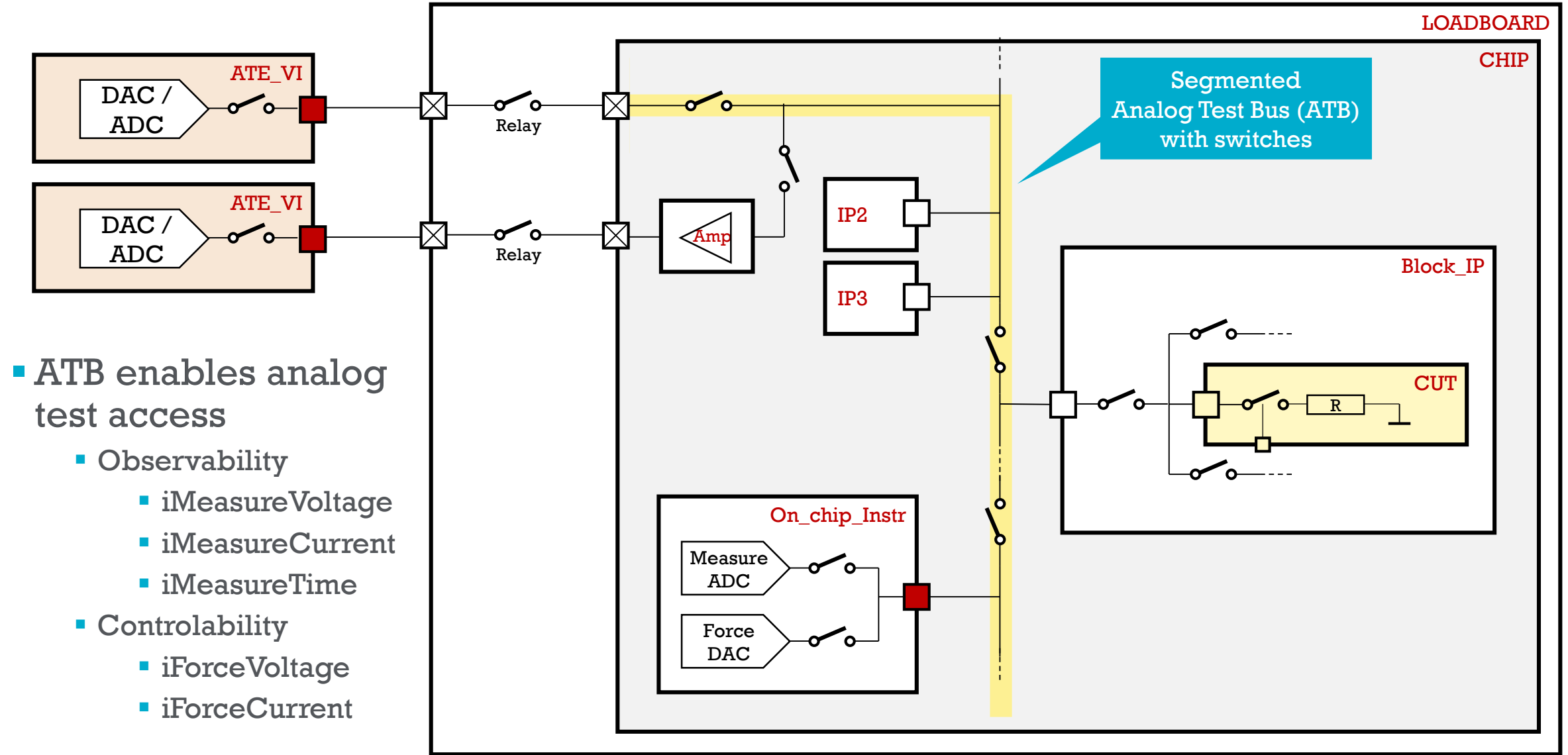
Analog Test Bus + Digital Core

- I2C
- GPIO



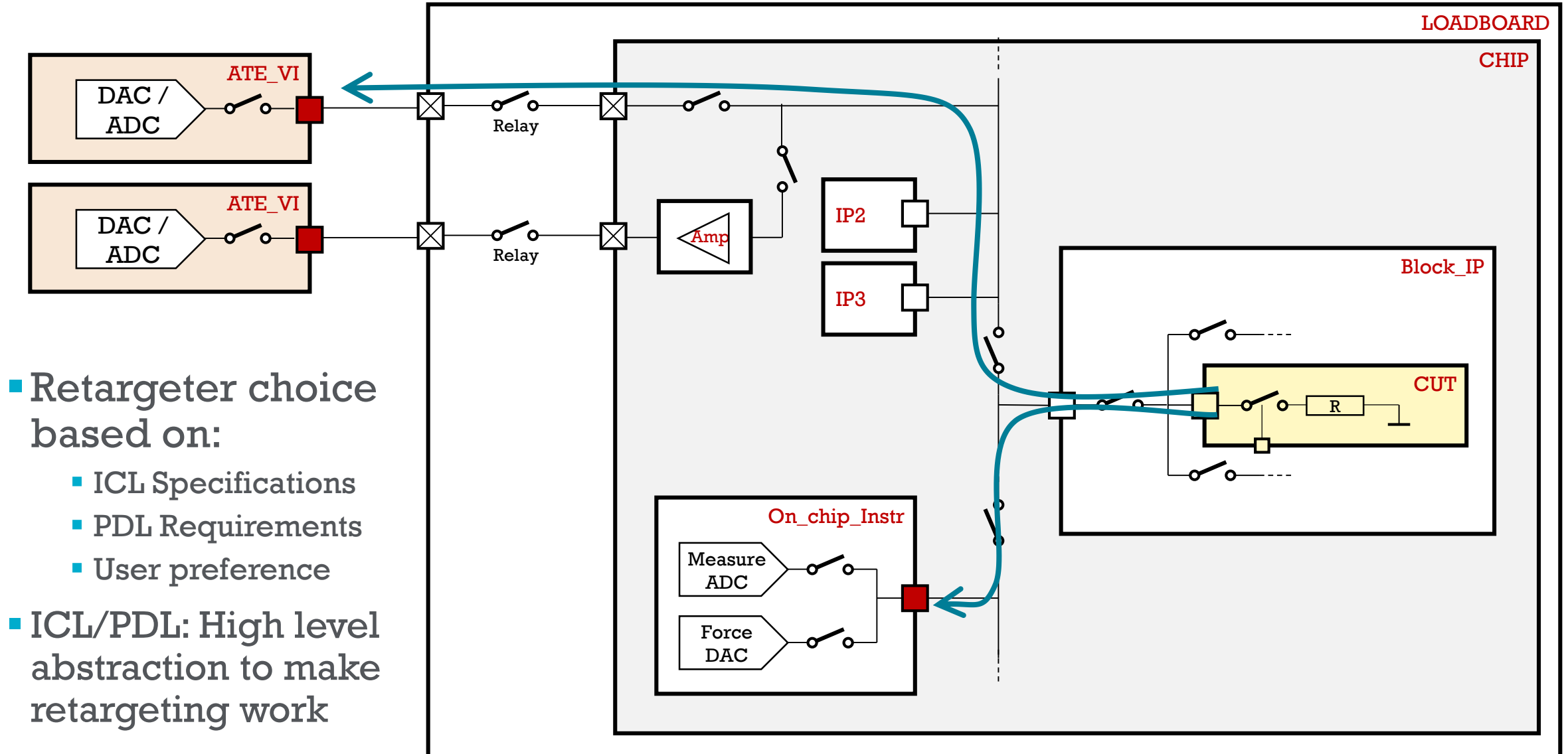
« Industrial Application of IJTAG Standards to the Test of Big-A/little-d Devices – plus Updates to the Latest State of IEEE P1687.2 », H. M. von Staudt, M. Benhebibi, J. Rearick, M. Laisne, Distinguished ITC 2020 Paper INTERNATIONAL TEST CONFERENCE ASIA

CUT Deeply Embedded in a Chip: Need Test Access



- ATB enables analog test access
 - Observability
 - iMeasureVoltage
 - iMeasureCurrent
 - iMeasureTime
 - Controlability
 - iForceVoltage
 - iForceCurrent

From CUT to Instrument: Retargeter Chooses Path



P1687.2 Retargeting Examples

TABLE I. REGISTERS: USER SPACE

Register	Bit Assignments							
	7	6	5	4	3	2	1	0
GPIO_CTRL 0x10	gpo1_en	gpo1_data	-	-	-	-	-	-
LD01_CTRL 0x20	ldo1_en	ldo1_pden	ldo1_vset[5:0] 0x00 = 1.2V, LSB=0.05V, 0x3F = 4.35V					

TABLE II. REGISTERS: TEST SPACE

Register	Bit Assignments							
	7	6	5	4	3	2	1	0
ATB_CTRL 0xF1	atb_en	atb_pden	-	-	-	-	-	-
DMUX_CTRL 0xF2	dmux_en	-	-	-	-	dmux_sel[2:0]		
LD01_TST 0xF8	ldo1_atben	-	-	-	ldo1_trim[3:0] 0x0=-80mV, LSB=10mV, 0xF=+70mV			

TABLE III. DIGITAL TEST MULTIPLEXER TABLE

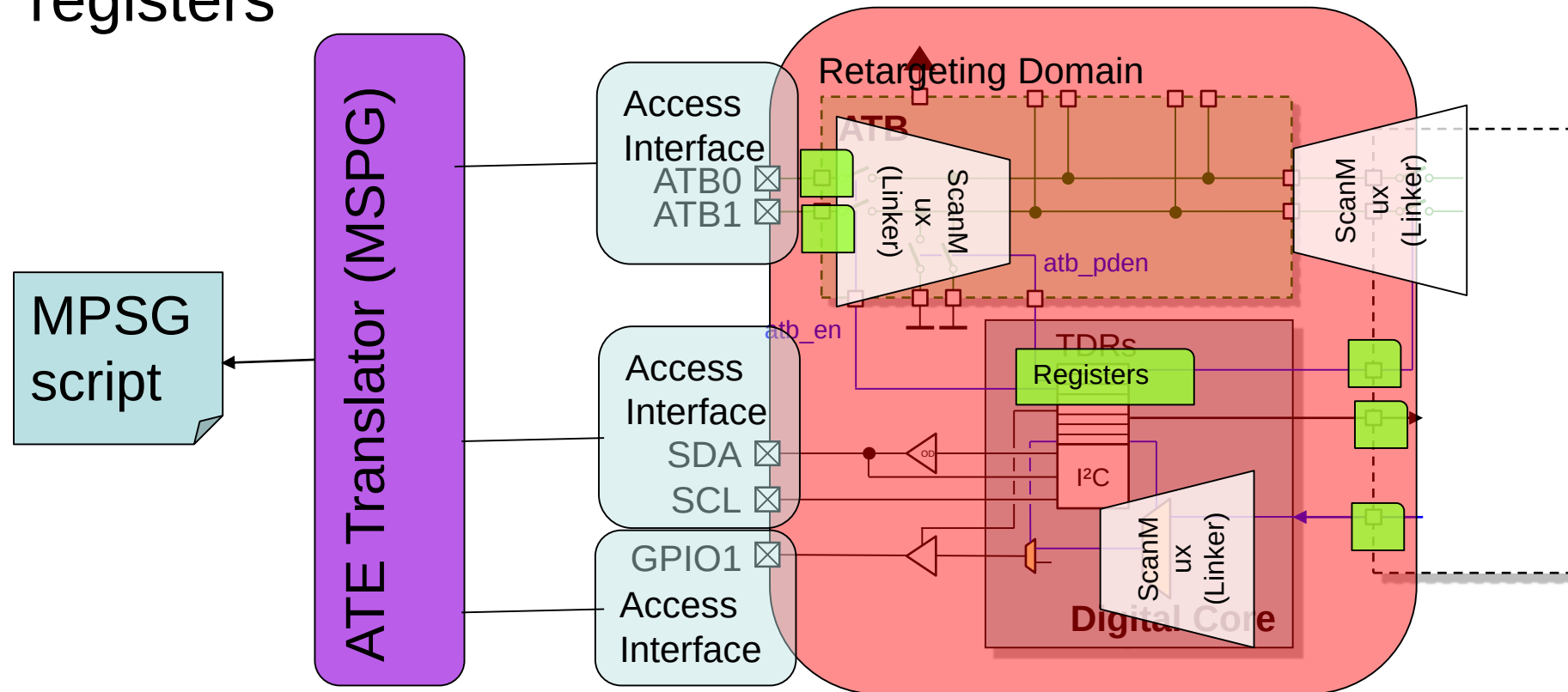
Selector dmux_sel	Selected Signal	Comment
3'b000	1'b0	For debug
3'b001	1'b1	
...	undefined	
3'b101	ldo1_startup	Startup signal
3'b110	ldo1_imit	Over current comparator
3'b111	ldo1_ov	Overvoltage comparator

TABLE IV. RETARGETING (R) AND TRANSFORMATION (T) FOR THREE SELECTED COMMANDS FROM THE LDO TEST PROCEDURE TO CHIP AND ATE LEVEL

Level	iWrite Command	iRead Command	iMeasure Command
LDO	iWrite en off; iWrite pden on; iApply;	iRead startup 0; iApply;	iMeasureVoltage vout_s vmeas; iApply;
DIG CORE (R)	iWrite ldo1_en 0; iWrite ldo1_pden 1; iApply; iWrite LD01_CTRL[7:6] 2'b01; iApply;	iWrite dmux_en 1; iWrite dmux_sel 3'b101; iApply; iRead gpo1_data 0; iApply;	
(R)	iWrite DIG.ldo1_en 0; iWrite DIG.ldo1_pden 1; iApply;	iWrite DIG.DMUX_CTRL 8'b1_0000_101; iwrite DIG.GPIO_CTRL[7] 1'b1; iApply; iRead GPIO1 0; iApply;	iWrite DIG.atb_en 1; iWrite DIG.atb_pden 0; iApply; iMeasureVoltage ATB1 vmeas; iApply;
PMIC (T)	write_i2c DIG.LD01_CTRL[7] 1'b0; write_i2c DIG.LD01_CTRL[6] 1'b1; iApply;	write_i2c DIG.DMUX_CTRL 8'b1_0000_101; write_i2c DIG.GPIO_CTRL[7] 1'b1; iApply; iRead GPIO1 0; iApply;	write_i2c DIG.ATB_CTRL[7] 1'b1; write_i2c DIG.ATB_CTRL[6] 1'b0; iApply; iMeasureVoltage ATB1 vmeas; iApply;
ATE (T)	' Select the interface to use ComInterface.SelectInterface I2C MSPG.Dig.Write rBF.ldo1_en, &H0 MSPG.Dig.Write rBF.ldo1_pden, &H1 MSPG.Dig.Write rREG.LD01_CTRL, &H40	MSPG.Dig.Write rREG.DMUX_CTRL, &H85 MSPG.Dig.Write rBF.dmux_en, &H1 MSPG.Dig.Write rBF.dmux_sel, &H5 ' Use Digital instrument comparator ' (expect "L" on GPIO1 pin) MSPG.Dig.Pin.Read "GPIO1", Compare.Low	MSPG.Dig.Write rBF.atb_en, &H1 MSPG.Dig.Write rBF.atb_pden, &H0; MSPG.DCVI.Strobe "ATB1" ' Capture using analog instrument

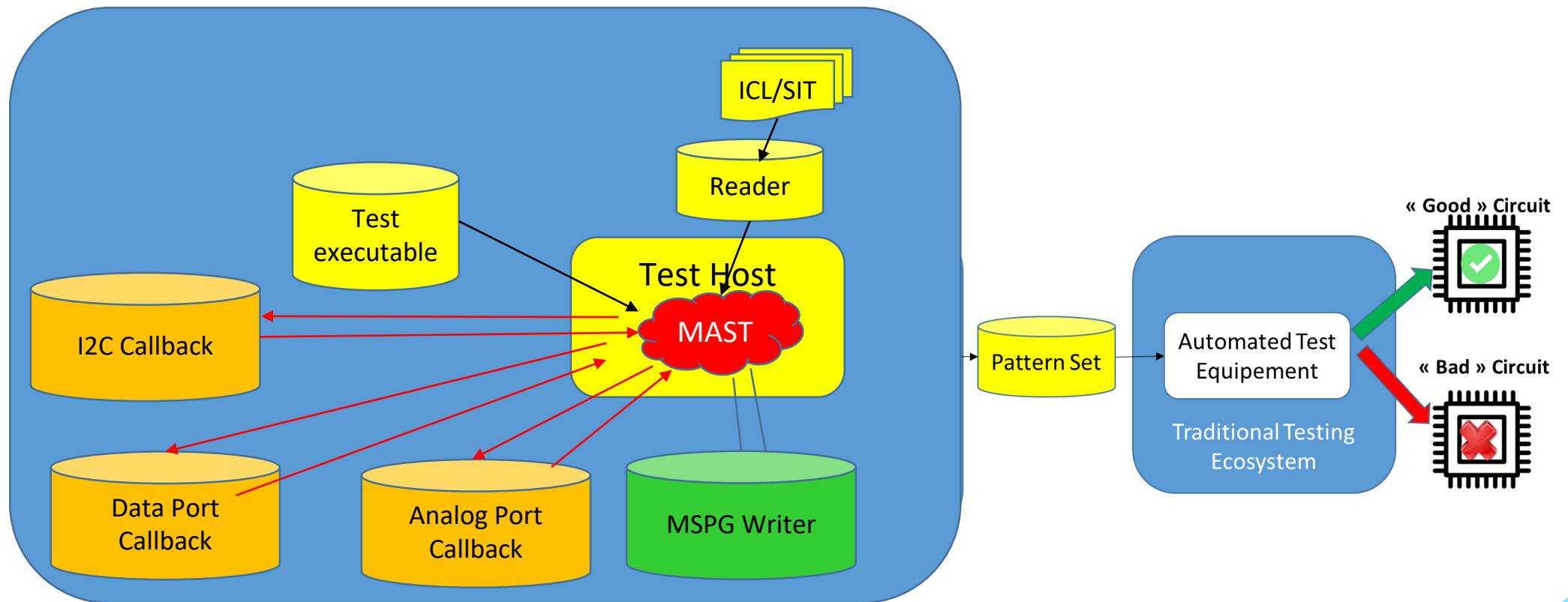
P1687.1 SIT Representation (1)

- Limitation: SIT cannot express ports (yet): modelled as 1-bit registers



MAST P1687.1 Usage Principles

- One callback per Interface (port) type
- MSPG Pattern Writer as a P1687.1 interface



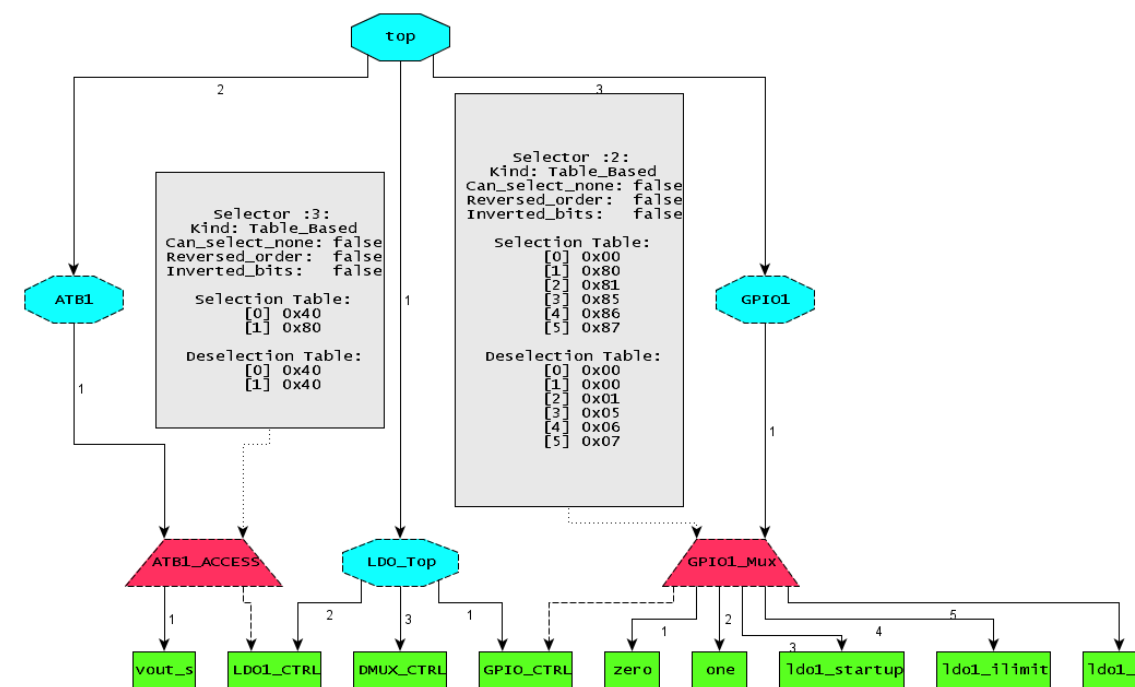
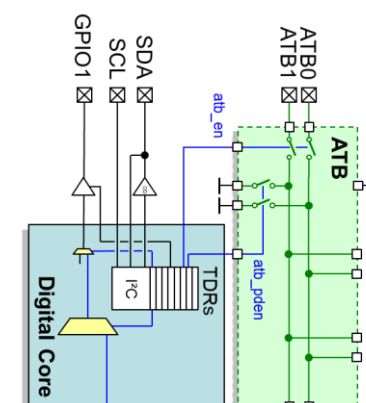
P1687.1 SIT Representation (2)

TRANSLATOR top MPSPG

```

PDL LDO_PDL
(
  ACCESS_INTERFACE LDO_Top NOT_IN_PATH I2C "0x00, 0x10, 0x20, 0xF2"
  (
    REGISTER GPIO_CTRL 8 Bypass: "0x00"
    REGISTER LDO1_CTRL 8 Bypass: "0x00"
    REGISTER DMUX_CTRL 8 Bypass: "0x00"
  )
  ACCESS_INTERFACE ATB1 NOT_IN_PATH AnalogPort "ATB0"
  (
    LINKER ATB1_ACCESS Table_Based LDO1_CTRL 1 "0b01000000, 0b10000000,
    0b01000000, 0b01000000"
    (
      REGISTER vout_s 8 Bypass: "0x00"
    )
  )
  ACCESS_INTERFACE GPIO1 NOT_IN_PATH DataPort "GPIO1"
  (
    LINKER GPIO1_Mux Table_Based GPIO_CTRL 5 "0b00000000, 0b10000000,
    0b10000001, 0b10000101, 0b10000110, 0b10000111,
    0b00000000, 0b00000001, 0b00000101, 0b00000110, 0b00000111 "
    (
      REGISTER zero 1 Bypass: "0b0"
      REGISTER one 1 Bypass: "0b1"
      REGISTER ldo1_startup 1 Bypass: "0b0"
      REGISTER ldo1_ilimit 1 Bypass: "0b0"
      REGISTER ldo1_ov 1 Bypass: "0b0"
    )
  )
)

```



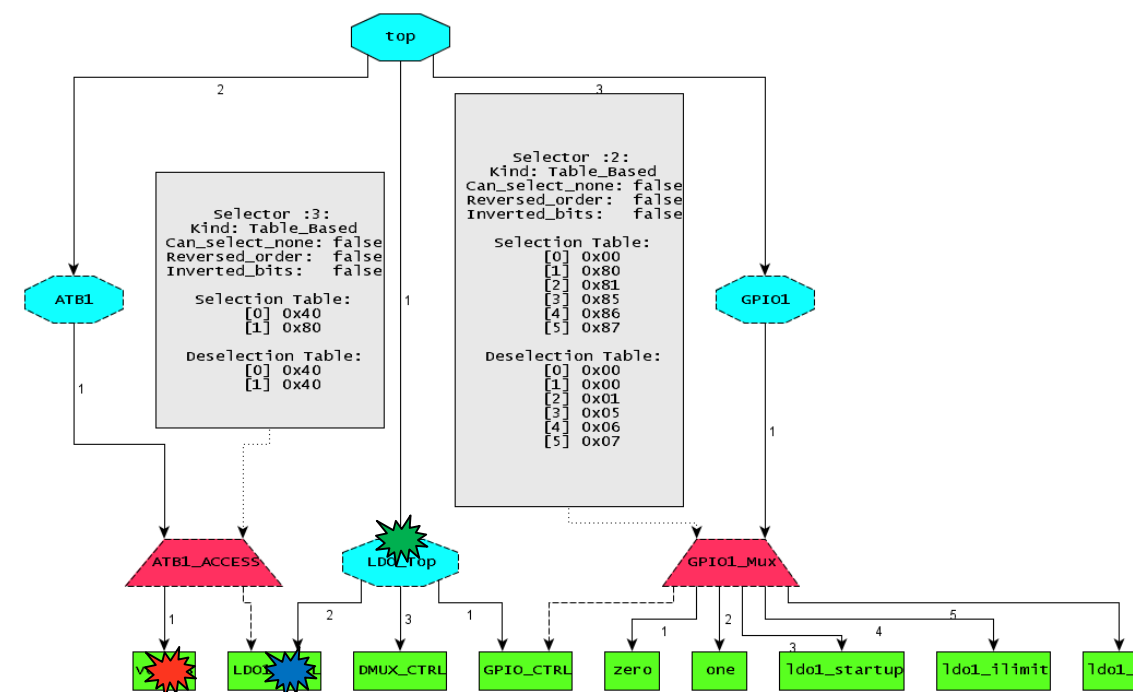
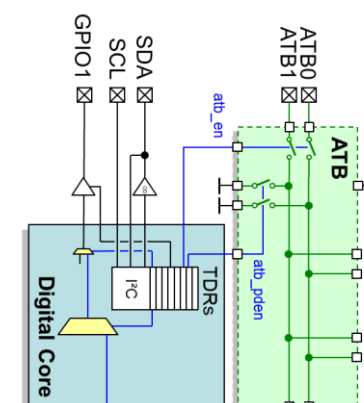
MAST Working principle

- 1) PDL is executed
 - 1.1 Instructions are queued into node, which become « pending »
`iWrite("vout_s", regvalue);`
- 2) iApply is a synchronization edge
`iApply();`
 → put PDL on hold
- 3) MAST « Data Cycle »


```
while (pending > 0)
{
  Propagate « pending » to top
    if needed, set new pending for topology configuration

  Retarget current active retargeting space
  Perform a RVF operation at the edge of retargeting space
  Propagate and transform RVF until top

  Delete « satisfied » pending from list
    if no more pending, release PDL
}
```



P1687.1 Data Port

- Callback: generate a RVF request on the port

```
BinaryVector DataPort_Protocol::DoCallback (uint32_t /*channelId*/, void* /*  
interfaceData */, const BinaryVector& toSutData)  
{  
    auto c_portname = m_portname.c_str();  
    RVFRequest request(DATAPORTREAD, (void *)c_portname);  
  
    PushRequest(request);  
  
    /*NB: this is a BLOCKING call*/  
    PopfromSut();  
  
    return toSutData;  
}
```

IDF	DATAPORTREAD
toSutData	Unused
InterfaceData	« GPIO1 »
ExpectedData	Not implemented

- Limitation: MAST checks mismatch in PDL, no « expected data » in RVF → could be added

P1687.1 Analog Port

- Callback: generate a RVF request on the port

```
BinaryVector AnalogPort_Protocol::DoCallback (uint32_t /*channelId*/, void* /*
interfaceData */, const BinaryVector& toSutData)
{
    auto c_portname = m_portname.c_str();
    RVFRequest request(IMEASUREVOLTAGE, (void *)c_portname);

    PushRequest(request);

    /*NB: this is a BLOCKING call*/
    PopfromSut();

    return toSutData;
}
```

IDF	IMEASUREVOLTAGE
toSutData	Unused
InterfaceData	« ATB0 »

- Limitation: MAST checks mismatch in PDL, no « expected data » in RVF → could be added

P1687.1 MSPG Protocol

- Issue correctly formatted MSPG commands to file

```
BinaryVector MSPG_TranslatorProtocol::TransformationCallback(RVFRequest current_request)
```

```
{
    [...]
    os << "MSPG.";

    //Check for known commands to format
    if (current_request.CallbackId()=="I2C_READ")
    {
        os << "DIG.Read.I2C";
        commandData = FormatI2CCommand(current_request);
    }
    else
    {
        if (current_request.CallbackId()=="I2C_WRITE")
        {
            os << "DIG.Write.I2C";
            commandData = FormatI2CCommand(current_request);
        }
        else
        {
            if (current_request.CallbackId()=="iMeasureVoltage")
            {
                os << "DIG.DVI.Strobe.";
                commandData = '\n';
                commandData += (char *) current_request.interfaceData();
                commandData += '\n';
            }
            else
            {
                if (current_request.CallbackId()==DATAPORTREAD)
                {
                    os << "DIG.DVI.Pin.Read ";
                    commandData = '\n';
                    commandData += (char *) current_request.interfaceData();
                    commandData += '\n';
                }
            }
        }
    }
    [...]
}
```

```
m_ofs << command << '\n'; //Log command into Emulation log
m_ofs.flush();
```

PDL : same old PDL

```
void LDO_Demo ()
{
    auto    registerPath = "LDO1_CTRL";
    auto    AnalogPort = "vout_s";
    auto    loopCount    = 5u;
    auto    i    = 0u;
    uint16_t initialValue = 1u;
    uint16_t curValue;

    uint8_t regvalue=0x0;

    regvalue = en_off+pden_on;

    std::cout << "\nStarting LDO Test\n";
    std::cout << "LDO off, Pullup on\n";
    iNote(iNoteType::Comment,"LDO off, Pullup on");
    iWrite("LDO1_CTRL", regvalue);
    iApply();

    std::cout << "ReadData from Analog Port\n";
    iNote(iNoteType::Comment,"ReadData from Analog Port");
    auto PortValue = iGetRefresh<uint8_t>(AnalogPort);
    iApply();

    std::cout << "ReadData from GPIO\n";
    iNote(iNoteType::Comment,"ReadData from GPIO");
    iRead("ldol_startup", 0x0);
    iApply();

    std::cout << "LDO Test Finished\n" ;
}
```

MSPG.DIG.Read.I2C(0x20,0x40);
MSPG.DIG.Write.I2C(0x20,0x40);
MSPG.DIG.Read.I2C(0x20,0x80);
MSPG.DIG.Write.I2C(0x20,0x80);
MSPG.DIG.DVI.Strobe."ATB0"
MSPG.DIG.Read.I2C(0x10,0x85);
MSPG.DIG.Write.I2C(0x10,0x85);
MSPG.DIG.DVI.Pin.Read "GPIO1"

NB: by default MAST always issues CSU
(read+write), can be optimized

P1687.2 Extensions

- ICL .2 information used to configure Translators
- Dedicated RVF primitives
- Retargeting after Transfer function?

